Balderrama Alexis.
Bergonzi Juan.
Inteligencia Artificial 2026 4to Ing. Informática.

PrestaMatch – Informe final

Selección de cartera de préstamos P2P con un presupuesto de USD 2.000.000

1. Cómo modelamos el problema

La plataforma recibe solicitudes de préstamo y solo puede financiar un subconjunto con USD 2.000.000. Cada préstamo, si se paga, rinde monto × tasa; si cae en default, se pierde el monto. Por eso el objetivo es la ganancia de la cartera y no la exactitud de una clasificación. Lo dividimos en dos partes: (1) un clasificador que estima la probabilidad de pago p de cada solicitud (Estado_Final: 1 = pagó, 0 = default) y (2) una decisión de cartera que, con esas probabilidades, elige qué aprobar sin exceder el presupuesto. La ganancia esperada de cada solicitud es monto × [p × (1 + tasa) − 1].

2. Datos y decisiones sobre las entradas

- **Cruce con el banco.** Se unieron las solicitudes con comportamiento_bancario.csv por ID_Usuario, y de cada usuario solo se usaron los 12 meses anteriores a la solicitud, para que ninguna variable contenga información posterior a la decisión (sin fuga temporal).
- **Variables.** Se resumió el historial en n_meses_hist, saldo_medio y atraso_medio, y se derivaron monto_sobre_ingreso (cuántos sueldos pide), puede_pagar (el saldo medio cubre el monto) e interes_potencial (monto × tasa). El modelo usa 9 variables numéricas.
- **Descartes.** Se eliminaron Motivo_Prestamo y Fecha_Solicitud. ID_Solicitud e ID_Usuario solo identifican registros: usarlos haría que el modelo memorice en vez de aprender.
- **Nulos y duplicados.** Los nulos se imputaron con la mediana calculada solo con train y aplicada igual a test. Los archivos finales (train_clean.csv: 32.416 filas; test_clean.csv: 3.061) quedan sin nulos y sin ID_Solicitud duplicados.
- **Desbalance.** El 84,05 % de los préstamos de train se pagó. Por eso todas las particiones son estratificadas, de modo que cada una conserve esa proporción.
- **Escala distinta en test.** El monto medio solicitado es de USD 14.290 en train y de USD 2.235 en test, y puede_pagar vale 1 en el 15,0 % de train y en el 79,2 % de test. Es un riesgo de generalización que dejamos documentado; las tasas, en cambio, son comparables.

3. Solución propuesta

Un Random Forest dentro de un pipeline de scikit-learn, de modo que el preprocesamiento se ajusta solo con los datos de entrenamiento. La versión final usa 200 árboles, profundidad máxima 8, mínimo 40 observaciones por hoja, max_features = 'sqrt' y semilla 42; la primera versión usaba profundidad 16 y mínimo 60 por hoja. Con las probabilidades se calcula la ganancia esperada y se resuelve un problema de mochila entera (MILP) que maximiza la ganancia esperada total sin superar el presupuesto, considerando solo solicitudes con ganancia esperada positiva. Un puntaje de reglas (ingreso, interés potencial y atraso, con pesos 1:1:1) actúa únicamente como desempate marginal. El archivo de salida (solicitudes_test.csv) tiene una decisión de aprobación por solicitud: la versión final aprobó 646 y usó USD 1.999.800 del presupuesto.

4. Validación y comparación de enfoques

Medimos siempre fuera de la muestra de entrenamiento, con hold-out estratificado 70/30 y K-Fold estratificado de 5 folds. La métrica es de negocio: monto × tasa si el préstamo se paga, −monto si hay default y 0 si se rechaza. Para comparar, el ROI se calcula sobre el presupuesto simulado, que en cada conjunto es el 29,2 % del monto pedido (la proporción que USD 2.000.000 representan en test).

Estrategia (mismo presupuesto relativo: 29,2 % del monto pedido)	Hold-out 30 %	CV 5 folds (media ± desvío)
Baseline: prestar primero a la tasa más alta	−3,72 %	−4,93 % ± 1,42
Regla de puntaje 1:1:1 (ingreso, interés potencial, atraso)	17,80 %	17,32 % ± 1,37
Árbol de decisión (profundidad 6) + optimización de cartera	22,39 %	22,11 % ± 0,42
Random Forest, primera versión (profundidad 16, hoja 60)	22,23 %	23,13 % ± 0,54
Random Forest final (profundidad 8, hoja 40)	**22,07 %**	**22,83 % ± 0,54**

Con el presupuesto fijo de USD 2.000.000, el Random Forest final obtuvo una ganancia neta de USD 446.709 en hold-out (ROI 22,34 % sobre el capital prestado) y de USD 529.756 en promedio en CV (ROI 26,49 %), positiva en los cinco folds (de USD 493.691 a USD 591.769). La primera versión obtuvo USD 449.628 (22,48 %) y USD 532.878 (26,64 %).

5. Otros enfoques probados

- **Baseline por tasa.** Prestar primero a quien paga la tasa más alta. Perdió dinero en validación (ROI −3,72 % en hold-out y −4,93 % en CV): la tasa por sí sola no distingue a los buenos pagadores.
- **Regla de puntaje sin modelo.** Aprobar por el promedio ponderado 1:1:1 de ingreso, interés potencial y atraso. Fue rentable (ROI ≈ 17 %) pero quedó 4,3 a 5,5 puntos por debajo del Random Forest final y con mayor variabilidad entre folds, por lo que la dejamos solo como desempate.
- **Árbol de decisión de profundidad 6.** Mismo esquema (probabilidad de pago + optimización de cartera) con un único árbol de profundidad 6 y mínimo 40 observaciones por hoja. Rindió casi igual en hold-out (22,39 % contra 22,07 %) y algo menos en CV (22,11 % contra 22,83 %). Con USD 2.000.000 fijos obtuvo USD 433.661 en hold-out y USD 515.844 en CV, y en test una ganancia esperada de USD 510.231 con 608 aprobadas, 469 de ellas en común con la primera versión del Random Forest. En la plataforma quedó muy por debajo de los Random Forest.
- **Random Forest, primera versión.** Con profundidad 16 y mínimo 60 por hoja. Redujimos la profundidad a 8 y el mínimo por hoja a 40: en validación el rendimiento fue equivalente (23,13 % contra 22,83 % en CV, dentro de la variabilidad entre folds) y la selección en test casi no cambió (607 de 649 aprobadas en común), pero en la plataforma la versión final mejoró levemente.

6. Resultado en la plataforma y limitaciones

La tabla de posiciones se calcula con aproximadamente el 30 % de los datos de test; el resultado final usará el 70 % restante. Subimos cuatro versiones:

Estrategia	Aprobadas	Ganancia esperada (test completo)	Plataforma (≈ 30 % del test)
Regla de puntaje 1:1:1	406	—	USD 131.310,75
Árbol de decisión (profundidad 6)	608	USD 510.231,03	USD 129.347,68
Random Forest, primera versión	649	USD 512.697,56	USD 171.368,56
Random Forest final (prof. 8, hoja 40)	646	USD 507.708,58	USD 172.695,35

Los puntajes de la plataforma corresponden a ≈ 30 % del test; las ganancias esperadas, a todo el test.

Las dos versiones del Random Forest fueron las mejores en la plataforma: la final obtuvo USD 172.695 y la primera USD 171.369. La diferencia, de USD 1.327 (menos del 1 %), es pequeña para una muestra parcial de ≈ 30 %. Ambas están en línea con lo esperado o por encima: el 30 % de la ganancia esperada para todo el test es de unos USD 152.300 en la versión final y de unos USD 153.800 en la primera. En cambio, el árbol de decisión quedó por debajo (unos USD 153.100 esperados contra USD 129.348) y la regla de puntaje, que en validación rendía menos que él, quedó apenas por encima. Con una muestra parcial no podemos afirmar si esas diferencias son ruido o reales; una posibilidad es que un único árbol, más ajustado a los datos de entrenamiento, se resienta por la diferencia de escala de los montos entre train y test, pero no lo comprobamos. Como la validación fue equivalente entre las dos versiones y la final obtuvo el mejor puntaje en la plataforma, adoptamos el Random Forest de profundidad 8 y mínimo 40 por hoja como solución final.